260602 Franka LLM Drawing Project Status Summary

IR / Franka LLM Drawing Project

2026-06-02

1 Purpose

This document explains the current code structure, Isaac Sim execution flow, demo status, and future development direction to project teammates. It focuses on the current state rather than every intermediate tuning step.

2 Current project status

The project currently reaches the stage where a natural-language-like drawing command is converted into a `DrawingPlan JSON`, passed through a JADE-based robot execution layer, and executed in Isaac Sim with a Franka robot carrying a pen.

This is not the final complete system yet. The current demo uses a reproducible API-free `no-api` fallback instead of paid OpenAI API calls.

```
natural-language-like input
-> no-api fallback planner
-> DrawingPlan JSON
-> bridge validator
-> JADE trajectory/control
-> Isaac Sim Franka execution
-> logs and plots
```

3 Folder roles

- `Unit_Action_Langchain-main/`: Existing LLM planner layer. Its role is natural language to DrawingPlan JSON. It does not handle IK, Jacobians, joint commands, or Isaac execution.
- `llm_isaac_bridge/`: Thin bridge added for this stage. It handles natural-language-like input, plan generation, validation, and Isaac command generation. The current presentation demo uses the API-free fallback.
- `franka_llm_drawing/`: Main robot-control core package currently used from the IR root. It handles trajectory generation, frame transforms, pen-tip offsets, DLS IK, normal-force admittance, JADE evaluation, and Isaac execution.
- `franka_llm_drawing_control/`: Self-contained copy prepared for team GitHub sharing.
- `configs/`: Frame, Isaac scene, JADE controller, and sampling configuration files.
- `usd/`: Isaac USD scene with Franka, table, paper, pen, and lights.
- `outputs/`: Execution results, validation reports, plots, and CSV logs.

4 Implemented core features

4.1 USD / Isaac scene

- The current scene is `usd/franka_drawing_scene_clean.usda`.

- Franka is referenced from a local project asset.
- The scene contains table, paper, pen, and lights.
- The paper surface is aligned with `/World/BoardFrame`.
- The controlled drawing target is `PenTipFrame`.

4.2 Commanded joints

The Isaac backend commands only the seven Franka arm joints:

```
panda_joint1, panda_joint2, panda_joint3, panda_joint4,
panda_joint5, panda_joint6, panda_joint7
```

Finger joints may still exist in the USD articulation, but they are not command targets.

4.3 Trajectory / Controller / JADE

- The system handles `move_to_start`, `pen_down`, `draw_line`, `draw_arc`, and `pen_up`.
- Lines and arcs are time-sampled with quintic time scaling.
- Speed, acceleration, and jerk limits are applied.
- The baseline controller is a joint-position backend with DLS IK, not a full torque controller.
- Normal-force admittance adjusts the paper-normal contact behavior.
- JADE records tracking error, tip orientation error, normal force, singular value, condition number, manipulability, and joint-limit margin.

5 How the simulation runs

1. The user enters a natural-language-like command.
2. `llm_isaac_bridge` runs the no-api fallback planner.
3. A DrawingPlan JSON file is generated.
4. The bridge validator checks action order, board bounds, speed, pen state, and line/arc continuity.
5. The existing `run_jade_isaac.py` runner samples the plan into a trajectory.
6. Board-frame targets are transformed into robot-base-frame targets.
7. Pen-tip offset compensation converts desired pen-tip pose into desired `panda_link7` pose.
8. The Isaac backend reads q, qdot, end-effector pose, Jacobian, and contact force.
9. The controller generates targets for `panda_joint1` through `panda_joint7`.
10. Isaac Sim steps forward and saves results under `outputs/`.

6 API-free presentation demo inputs

case id	input meaning	output
`circle_r4cm`	circle with 4 cm radius at the center	circle
`square_s6cm`	square with 6 cm side length at the center	square
`triangle_s6cm`	triangle with 6 cm side length at the center	triangle
`letter_a_8cm`	letter A with 8 cm size at the center	letter A
`house_8cm`	8 cm house shape at the center	house
`star_8cm`	8 cm star at the center	star

These are deterministic rule-based fallback cases for a stable presentation demo, not true LLM reasoning results. The real LLM planner can be swapped in through the same DrawingPlan schema.

7 How to run the demo

Generate plans and validation reports:

```
cd /home/kimchangyeol/IsaacLab/IR
/home/kimchangyeol/IsaacLab/IR/llm_isaac_bridge/scripts/prepare_demo_showcase.sh
```

Run one case in Isaac:

```
cd /home/kimchangyeol/IsaacLab/IR
/home/kimchangyeol/IsaacLab/IR/llm_isaac_bridge/scripts/prepare_demo_showcase.sh \
  --case circle_r4cm \
  --execute-isaac
```

Generated files:

```
outputs/demo_showcase/plans/*.json
outputs/demo_showcase/reports/*_validation.json
outputs/demo_showcase/manifest.json
outputs/demo_showcase/isaac_commands.sh
```

For video recording, run one case at a time and manually record the Isaac Sim window.

8 What works now and current limitations

8.1 Works now

- API-free natural-language-like input
- DrawingPlan JSON generation and validation
- Demo plans for circle, square, triangle, A, house, and star
- Isaac Sim execution of Franka pen-tip trajectories
- Contact force, tracking error, orientation error, and Jacobian metric logging
- Result plots and summary generation

8.2 Current limitations

- The no-api fallback is not a real LLM.
- It does not understand arbitrary natural language.
- The current fallback mostly supports centered drawings.
- Ink rendering on paper is not implemented yet.
- Controller performance is still being improved.
- Real robot hardware execution is not implemented.

9 Recommended future advanced features

- Connect a real OpenAI API or local LLM planner.
- Support more shapes, composed drawings, and location phrases.
- Add stronger schema validation for LLM output.
- Add automatic corner slowdown, stroke-order optimization, and robot reachability pre-check.
- Reduce contact force spikes and improve guarded pen-down.
- Improve tangent tracking correction and adaptive speed scaling.
- Add optional torque hybrid force-position control as an advanced mode.
- Add automatic camera pose, screen recording, ink trail rendering, and report generation.

10 Main message for teammates

1. The robot-control, Isaac scene, bridge, and JADE execution layers after the original LLM planner are now implemented separately.
2. The current system can show an API-free natural-language-like drawing demo in Isaac Sim.
3. Real LLM integration is structurally ready, but the presentation demo uses no-api fallback for cost and reproducibility.
4. The controller is not final; contact force and trajectory tracking will continue to improve.
5. The next discussion should prioritize real LLM integration, more shapes, stronger force control, automatic recording, or automated reports.